

컨테이너·포트 — 만들기·켜기·연결

도커 실습에서 가장 자주 만나는 오류입니다. 오류 문구로 찾으세요.

No such container: de-mysql

어디서: 2-1 실습점검 4단계 · 2-2 시작 (이번 수업 중 수강생 질문)

원인: 그 이름의 컨테이너가 아직 없거나(만들기 전에 exec-start 를 친 것) 앞 차시에서 지웠습니다. 틀린 것이 아니라 아직 만들기 전이라는 뜻입니다.

해결: 노트의 순서대로 docker run 으로 먼저 만듭니다. 있는지 확인은 아래 명령으로 합니다.

```
docker ps -a --filter name=de-mysql
```

bash

The container name "/web" is already in use (Conflict)

어디서: 1-3 컨테이너 기초 · 2-1 포트를 바꿔 다시 띄울 때

원인: 같은 이름의 컨테이너가 멈춘(exited) 채로 남아 있습니다. 멈춘 컨테이너도 이름을 붙들고 있습니다.

해결: 확인하고 지운 뒤 다시 만듭니다. 데이터는 볼륨에 있으면 그대로 남습니다.

```
docker ps -a
docker rm 이름          # 돌고 있으면 docker rm -f 이름
```

bash

port is already allocated · Ports are not available

어디서: 1주차 · 2-1 · 2-2 · 4주차 MinIO · 5주차 Spark · 7주차 · 8주차 Airflow

원인: 그 포트를 다른 컨테이너(앞 차시에 띄운 것)나 내 컴퓨터에 설치된 프로그램이 이미 쓰고 있습니다.

해결: 누가 쓰는지 보고 멈추거나, 내 쪽 포트 번호만 바꿉니다(-p 3307:3306 처럼 앞 숫자).

```
docker ps
lsof -nP -iTCP:3306 -sTCP:LISTEN          # 앞 차시 컨테이너가 떠 있는지
                                           # macOS·리눅스 - 포트 번호만 바꿔 씁니다
```

bash

Windows PowerShell 은 netstat -ano | findstr ":3306" 으로 봅니다. Windows 는 재부팅하면 멈춰 둔 MySQL80 서비스가 다시 켜질 수 있습니다(2-2).

켜자마자 접속하면 실패한다

어디서: 2-1 MySQL · 2-3 MongoDB · 6주차 Kafka

원인: 컨테이너가 Up 이어도 안의 서버는 아직 준비 중입니다. 준비 완료 표시를 보고 접속합니다.

해결: 서버마다 준비 완료 표시가 다릅니다.

```
# MySQL - 끝에 port: 3306 이 붙은 ready for connections 줄 (처음엔 port: 0 이 먼저 한 번 찍힙니다 - bash  
아직 아님)  
docker logs --tail 5 de-mysql  
  
# MongoDB - 로그 문구 대신 실제로 접속해 봅니다. 1 이 나오면 준비 완료  
docker exec de-mongo mongosh --quiet --eval "db.runCommand({ ping: 1 }).ok"  
  
# Kafka - "Kafka Server started" 줄  
docker logs kafka
```

MongoDB 의 「Waiting for connections」를 tail 3줄에서 찾는 안내가 많은데, 이 버전은 그 뒤에 줄이 더 붙어 보이지 않습니다(2-3 실측).

Unknown MySQL server host 'host.docker.internal' (ERROR 2005)

어디서: 2-1 기동과 접속 4단계 (이번 수업 중 수강생 질문)

원인: host.docker.internal 은 컨테이너에서 내 컴퓨터를 가리키는 이름입니다. Docker Desktop(macOS·Windows)은 자동으로 넣어 주지만, 리눅스나 WSL2 우분투에 도커 엔진만 설치한 경우에는 없습니다. 설정이 빠진 것이 아닙니다.

해결: --add-host 로 직접 넣어 줍니다. Docker Desktop 에서는 붙여도 결과가 같으니 항상 붙여도 됩니다.

```
docker run --rm --add-host=host.docker.internal:host-gateway mysql:8.0 \  
mysql -h host.docker.internal -P 3306 -uroot -p -e "SELECT VERSION();" bash
```

pull access denied for minio/minio, repository does not exist

어디서: 4주차 MinIO (2026-10-02 실측 — 수업 전 발견)

원인: MinIO 공식 이미지 minio/minio · minio/mc 를 Docker Hub 에서 더 이상 내려받을 수 없습니다(404).

해결: 같은 MinIO 를 빌드한 pgsty/minio · pgsty/mc 로 이름만 바꿉니다. 과정 노트는 이미 바꿔 두었습니다. 옛 자료를 따라 하다 이 오류가 나면 compose 의 image 줄을 확인합니다.

```
docker pull pgsty/minio  
docker pull pgsty/mc bash
```

grep: command not found · `sh` is not a recognized command (mc 컨테이너)

어디서: 4~6주차 mc 명령

원인: mc 이미지에는 grep·awk·sed·wget·curl 이 없고, 이미지가 mc 명령으로 시작합니다(2026-10-03 실측).

해결: 거르기(grep 등)는 컨테이너 밖에서 하고, docker run 으로 셸을 쓰려면 --entrypoint sh 를 줍니다.

```
docker compose exec -T mc mc ls --recursive local/warehouse | grep -c .parquet  
docker run --rm --network week6_default --entrypoint sh pgsty/mc -c "mc alias set m http://min  
io:9000 minioadmin minioadmin && mc ls m" bash
```

컨테이너 안에서 localhost 로 붙으면 Connection refused · UnknownHostException

어디서: 5주차 Spark ↔ MinIO

원인: 컨테이너 안에서 localhost 는 그 컨테이너 자신입니다. 다른 컨테이너는 같은 compose 네트워크 안에서 서비스 이름으로 찾습니다.

해결: 서비스들을 한 docker-compose.yml 에 두고, 주소를 서비스 이름으로 적습니다(예: http://minio:9000). 내 컴퓨터 (호스트)에서 붙을 때만 localhost 를 씁니다.

데이터 엔지니어링 & RAG 파이프라인 · 부록 — 서버 주소(SERVER_IP)·계정(devuser)·비밀번호(CHANGE_ME...)는 자리표시입니다. 본인 환경의 값으로 바꿔 쓰세요.